Homework1.HO1.Python Primer

Important

Save this file with the name ending with your initials and last two digits of your ID number, i.e. you will name the file "Homework1_HO1_JT88.pdf".

In addition, put a comment at the beginning of each code block to explain it with the same initials and digits. I have put one at the beginning as an example for you to follow. Do not forget to change it.

Finally, you will name all variables with your initials and last two digits of your ID: for instance, x = 3 below will be modified as xJT88 = 3.

If your initials and digits appear in any other student's submission or if another student's initials and digits appear in your submission you will be considered violating the ADTA academic integrity policy and subsequent penalty will be followed according to the syllabus.

Be sure to run all the codes and print to PDF before you submit your assignment.

1. Comments

Comments help a lot to take notes while coding. They are so useful to remind you and other coders of what you are doing.

Run the following code by simply clicking the "Run" button on the toolbar

In [1]:
```
# VB67 This is a simple comment to exemplify how the comments are placed in
# This is a comment.
```

Write a comment below to express your first impression about this course and run it. Remember that you are supposed to put a comment at the top of each code block with your initials and digits

In [4]:
```
# VB67
# My first impression about the course
print("My first impression about the course was very exciting and challengir
```

```
My first impression about the course was very exciting and challenging given
the mathematics involved behind the scenes of Machine Learning concepts.
```

2. Print text on screen

In [2]:
```
#VB67
print ("Hello World")
```

```
Hello World
```

Write a command below to print "Hello Professor Tong!"

In [3]:
```
# VB67
# printing "Hello Professor Tong!" on screen
print ("Hello Professor Tong!")
```

```
Hello Professor Tong!
```

3. Variables and Data Types

Variables are data containers whose values can change in the course of executing a program.

Variable/Identifier x:

- ***x*** is an ***identifier***. It is a ***variable.***
- Its ***value*** is 3: 3 is ***data***.
- The ***variable (identifier)*** x is the ***data container*** that contains the piece of data "3".
- ***Data type*** of x is "int".

Variable/Identifier stdName:

- ***stdName*** is an ***identifier***. It is a ***variable***.
- Its ***value*** is "Kate Winslet": "Kate Winslet" is ***data***.
- The ***variable (identifier)*** stdName is the ***data container*** that contains the piece of data "Kate Winslet".
- ***Data type*** of stdName is "str" (or String).

IMPORTANT NOTES:

- In Python, identifiers are unlimited in length. Case is significant.
- However, it is strongly discouraged from using a very long name to label variables. Besides, the names should be meaningful.

Here is an example

Run the following code:

In [4]:
```
# x is a name of a variable. x is an identifier.
x = 3

print ("x is a variable. It is an identifier. Its value is: ",x, "\n")
print("Data type of x: ", type (x), '\n')

#stdName is a name of a variable that represents the name of a student.
#stdName is an identifier.

stdName = "Kate Winslet"

print ("stdName is a variable. It is an identifier. Its value is: ", stdName
print("Data type of stdName: ", type (stdName), '\n')
```

```
x is a variable. It is an identifier. Its value is:  3

Data type of x:  <class 'int'>

stdName is a variable. It is an identifier. Its value is:  Kate Winslet

Data type of stdName:  <class 'str'>
```

Define another variable y and set it equal to last three digits of your ID number. Define another variable for student name (call it stname) and set it equal to your full name. Modify the block code above with these two new variables (y and stname) and run it. Be careful about naming variables explained at the beginning, i.e. y should appear as yJT88, for instance.

In [5]:
```
# VB67
# yVB67 is a name of a variable. yVB67 is an identifier.
yVB67 = 167

print ("yVB67 is a variable. It is an identifier. Its value is: ",yVB67, "\r
print("Data type of yVB67: ", type (yVB67), '\n')

#stname is a name of a variable that represents the name of a student.
#stname is an identifier.

stname = "Varshini Boyapally"

print ("stname is a variable. It is an identifier. Its value is: ", stname,
print("Data type of stname: ", type (stname), '\n')
```

```
yVB67 is a variable. It is an identifier. Its value is:  167

Data type of yVB67:  <class 'int'>

stname is a variable. It is an identifier. Its value is:  Varshini Boyapally

Data type of stname:  <class 'str'>
```

4. Assignments

Overview

The assignment operator, denoted by the "=" symbol, is the operator that is used to assign values to variables in Python.

We use Python assignment statements to assign objects to names. The target of an assignment statement is written on the left side of the equal sign (=), and the object on the right can be an arbitrary expression that computes an object.

Binding a variable in Python means setting a name to hold a ***reference*** to some object.

- Assignments create references, not copies.

Names in Python do not have an intrinsic type. ***Objects have types***.

- Python determines the ***type of the reference*** automatically based on the ***data object*** assigned to it.

A name is created when it appears the first time on the left side of an assignment expression:

> x = 3

A ***reference is deleted*** via garbage collection when NO variables or identifiers refer to it.

4.1. Reference Semantic in Python

Overview: The process of assigning a value

What happens when we type: x = 3?

- First, an integer 3 is created and stored in memory.
- Then a name is created.
- Next, a reference to the memory location storing the 3 is assigned to the name x

4.2 Assignment manipulates references

- y = x: An assignment of x to y
- y = x: The assignment does not make a copy of the value of x.
- y = x: The assignment makes y refer to the same object as x does.

> Let's say you assign x = 3

> And then you assign y = x

When you assign y = x, you now have y = 3 since x = 3.

Here is an example

Run the following code:

In [6]:
```
#VB67
#assign x=3 and x to y
x=3
y=x

#output y
y
```

Out[6]: 3

Revise the code block above with xx replacing x and set equal to 12.98, and yy replacing y . Don't forget to add your specifics to the variables, e.g. xxJT88

In [7]:
```
# VB67
xxVB67=12.98
yyVB67=xxVB67
yyVB67
```

Out[7]: 12.98

5. Operators and Operations

5.1 Numeric Operations

Operator	Operation	Example	Result
+	Addition	33 + 3	36
-	Subtraction	33 - 3	30
*	Multiplication	33 * 3	99
/	Division	11 / 3	3.666
%	Remainder	33 % 3	0

Operator	Operation	Example	Result
**	Exponent	33 ** 3	35937
//	Floor	11 // 3	3

Insert separate code blocks below to run all the examples above with different randomly selected integers. I am inserting the first line below for an example, you do the rest similarly.

In [8]:
```
# JT88 Numeric Operations
833 + 21
```

Out[8]: 854

In [9]:
```
# VB67 Numeric Operations(subtraction)
550 - 20
```

Out[9]: 530

In [10]:
```
# VB67 Numeric Operations(Multiplication)
50 * 21
```

Out[10]: 1050

In [11]:
```
# VB67 Numeric Operations(Division)
80 / 15
```

Out[11]: 5.333333333333333

In [12]:
```
# VB67 Numeric Operations(Remainder)
10 % 15
```

Out[12]: 10

In [13]:
```
# VB67 Numeric Operations(Exponent)
5 ** 7
```

Out[13]: 78125

In [14]:
```
# VB67 Numeric Operations(floor)
8 // 2
```

Out[14]: 4

5.2 Augmented Operators

Operator	Operation	Example	Result
+=	Addition assignment	i += 8	i = i + 8

Operator	Operation	Example	Result
-=	Subtraction assignment	i -= 8	i = i - 8
*=	Multiplication assignment	i *= 8	i = i * 8
/=	Division assignment	i /= 8	i = i / 8
%=	Remainder assignment	i %= 8	i = i % 8
**=	Exponent assignment	i **= 8	i = i ** 8
//=	Floor assignment	i //= 8	i = i // 8

Insert separate code blocks below to run all the examples above with different randomly selected integers. I am inserting the first lines below for an example, you do the rest similarly.

In [15]:
```
# JT88 Augmented Operators
i = 3
i += 12
i
```

Out[15]: 15

In [18]:
```
# VB67 Augmented Operators(Subtraction assignment)
i = 15
i -= 10
i
```

Out[18]: 5

In [19]:
```
# VB67 Augmented Operators(multiplication assignment)
i = 10
i *= 5
i
```

Out[19]: 50

In [20]:
```
# VB67 Augmented Operators(Division)
i = 7
i /= 2
i
```

Out[20]: 3.5

In [21]:
```
# VB67 Augmented Operators(Remainder)
i = 8
i /= 2
i
```

Out[21]: 4.0

In [23]:
```
# VB67 Augmented Operators(Exponent)
i = 6
i **= 3
i
```

Out[23]: 216

In [24]:
```
# VB67 Augmented Operators(Floor)
i = 12
i //= 3
i
```

Out[24]: 4

5.3 Relational Operators (a.k.a. Comparison Operators)

- Relational operators, a.k.a. comparison operators, are used for comparisons.
- A comparison, e.g. a>b, is called a conditional expression or boolean expression.
- The result of a boolean expression is either True or False.

Operator	Meaning	Example	Ex. Result
(<)	Less than	radius (<) 0	False
(=<)	Less than or equal to	radius (<=) 0	True
(>)	Greater than	radius (>) 0	True
(>=)	Greater than or equal to	radius (>=) 0	False
(==)	Equal to	radius (==) 0	True
(!=)	Not equal to	radius (!=) 0	False

Insert separate code blocks below to run all the examples above after setting the radius to a randomly selected number, say 4. I am inserting the first lines below for an example, you do the rest similarly.

In [26]:
```
# JT88 Relational Operators
radius = 4
radius < 0
```

Out[26]: False

In [29]:
```
# VB67 Relational Operators
radius = 50
radius <= 6
```

Out[29]: False

In [30]:
```
# VB67 Relational Operators
radius = 42
radius > 18
```

Out[30]: True

In [31]:
```
# VB67 Relational Operators
radius = 65
radius >= 59
```

Out[31]: True

In [32]:
```
# VB67 Relational Operators
radius = 82
radius == 82
```

Out[32]: True

In [33]:
```
# VB67 Relational Operators
radius = 50
radius != 10
```

Out[33]: True

5.4 Logical Operators

Operator	Meaning	Example
not	Opposite	not (radius < 0)
and	And	(a > b) and (c < d)
or	Or	(a > b) or (c < d)

5.5 Identity Operators

In Python, ***identity operators*** are used to check if the ***operands are identical***, i.e., they refer to the same object.

Operator	Result	Example
is	True if the operands are identical, i.e., they refer to the same object;	x is y
is not	True if the operands are not identical, i.e., they do not refer to the same object	x is not y

Let's illustrate what we have discussed.

Run the following 3 code blocks:

In [8]:
```
#VB67

x = 5
y = 5

isTrue = "x is y"
isFalse = "x is not y"
#check is x is y and print
if (x is y):
    print (isTrue)
else:
    print (isFalse)
```

```
x is y
```

In [6]:
```
#VB67

x = 5
y = 8

isTrue = "x is y"
isFalse = "x is not y"
#check is x is y and print
if (x is y):
    print (isTrue)

else:
    print (isFalse)
```

```
x is not y
```

In [7]:
```
#VB67
x = 5
y = 5

isTrue = "x is y"
isFalse = "x is not y"
#check is x is not y and print
if (x is not y):
    print (isTrue)
else:
    print (isFalse)
```

```
x is not y
```

5.6 Membership Operators

In Python, many data structures have their internal structure of a sequence, e.g., String, List, Tuple, etc.

For example:

- aString = "Hello World"
- aList = [1,2,3,4,5]
- aTuple = (1, 2, 3, 4, 5)

Membership operators - in, not in - are used to test ***if a value is found*** in a sequence or not.

Operator	Result	Example
in	True if the value/variable is found in the sequence;	x in y
not in	True if the value/variable is not found in the sequence	x not in y

Run the following 2 code blocks. Never forget to put comments on top of each such code block:

In [9]:
```
#VB67
x = 'Hello World'
#check if character H is in x
if ('H' in x):
    print ("H in x")
else:
    print ("H not in x")
```

```
H in x
```

In [10]:
```
#VB67
aList = [1, 2, 3, 4, 5]
# print if 8 is not in list "aList"
if (8 not in aList):
    print ("8 not in aList")
else:
    print ("8 in aList")
```

```
8 not in aList
```

6. Pseudo-Code

Overview

In machine learning, many developers write pseudo-code for their machine learning algorithms. Python pseudocode is a syntax-free representation of code. So, the Python pseudocode does not involve any code in it. The Python pseudocode is a very close

representation of the algorithmic logic. Read through the following scenario to better understand the purpose of pseudo-code.

6.1 Scenario: A Problem

It is assumed that a software developer is asked to write a Python program that can calculate and print the diameter and the circumference of a circle. The user enters the data of the radius and its measurement unit (in, ft, cm, or m) from the console.

6.2 How to Solve the problem

- First, we need to write pseudo-code (or create a flow-chart) of steps that can solve the problem. Formally, we design an algorithm that offers a solution to the problem.
- Then, based on the steps of the pseudo-code/flowchart (or algorithm), we write the code of the program.

These two steps are the foundation of the software engineering process.

6.3 Pseudo-Code

With pseudo-code, the developer writes down the steps to solve the problem in plain English.

1. Start
2. Read the input of the radius from the console
3. Read the measurement unit of the radius (in, ft, cm, m)
4. Calculate the diameter of the circle
    - diameter = 2*radius
5. Calculate the circumference of the circle
    - Circumference = diameter*PI (3.14159)
6. Print out the diameter
7. Print out the circumference
8. End

Suppose that you ask a person the year of his/her birth to compute the age of that person. Write a pseudo-code for this simple scenario below

1. Start
2. Read input of the Year of Birth from the user(him/her)
3. Read the Current Year from user
4. Calculate the age of the person in years
    - age = Current Year - Year of Birth
5. Print out the age of person in years
6. End

Great Job!

You have learned the fundamentals of programming for data science with Python.